

Saltzer and Schroeder's Principles of Security

Two senior members of the IEEE, J.H. Saltzer and M.D. Schroeder, wrote a paper titled

- “The Protection of Information in Computer Systems.”
- In this tutorial paper, Saltzer and Schroeder write in depth on architectural design and best practice principles for protecting information within information systems.
- The Protection of Information in Computer Systems” was published in 1975,
- This pioneering work is still surprisingly relevant to information security

Saltzer and Schroeder's Principles of Security

- **Security by Design**

- Security should not be an afterthought or add-on.
- When developing systems, you should begin with identifying relevant security requirements and treat them as an integral part of the overall process and system design.
- Begin with establishing and adopting relevant principles and policies as a foundation for your design, then build security into your development life cycle.
- We will discuss later Software development lifecycle (SDLC) where we can see security requirements can be incorporated in the design

Saltzer and Schroeder's Principles of Security

- **Security by Default**
- Secure by default means that the default configuration settings are the most secure settings possible.
- This is not necessarily the most user-friendly settings.
- Evaluate what the settings should be, based on both risk analysis and usability tests.

Saltzer and Schroeder's Principles of Security

- **Defense in Depth:** Also known as layered defense,
- Defense in depth is a security principle where defense against attack is provided by multiple security controls.
- To compromise a system or to break the security properties of the system-a sequence of activities (a sequence of attack vectors) may have been applied. Security control can be applied to stop the application of each vector. Therefore, multiple layers of control are applied.
- The aim is that single points of complete compromise are eliminated or mitigated by the incorporation of a series or multiple layers of security safeguards and risk-mitigation countermeasures.
- Example:
 - Physical control layer: key card
 - Access control Layer: User ID and password, MFA, Biometric, Authorization to resources by administrative control
 - Network security control: Firewall, VPN, IPS/IDS, Anti malware, Data leak protection tools, controls

Saltzer and Schroeder's Principles of Security

- **Fail Safe**

- This is a security principle that aims to maintain confidentiality, integrity and availability when an error condition is detected.
- Design systems so that if an error is detected, the system fails in a deny (or safe) state of higher security.
- For example unless an entity is given explicit access to an object, it should be denied access to that object by default.
- This is often described as 'Fail Safe Defaults' or 'Secure by Default'
- This is the principle behind “Deny All” default firewall rules.

Saltzer and Schroeder's Principles of Security

- **Least Privilege**

- A security principle in which a person or process is given only the minimum level of access rights (privileges) that is necessary for that person or process to complete an assigned operation.
- This right must be given only for a minimum amount of time that is necessary to complete the operation.
- **Example: with the Linux *sudo* command,**
- **This temporarily** elevates a user's permission to perform a privileged operation/execution of the command.
- This allows the user to perform a task requiring additional permissions only for the period of time necessary to perform the task.
- This is especially valid when it is restricted to privileged accounts related to specific services, but not used to access the root account.
- The increased security comes from not allowing the user to operate for extended periods with unneeded permissions.

Saltzer and Schroeder's Principles of Security

- **Compartmentalize**

- The compartmentalization principle helps in minimizing the impact of a security breach in case of a successful breach attempt
- The idea is to design the whole to consist of smaller connected parts.
- Compartmentalize the access to information on a "need-to-know" basis in order to perform certain tasks
- But must be used in moderation in order to prevent the system from becoming unmanageable
- Example: Role based access control (RBAC),
- Dividing a large database into smaller databases based on need-to-know which are independently managed and authorized access based on RBAC, if one small database is compromised , others are still safe.

Saltzer and Schroeder's Principles of Security

- **Separation of Duties**

- Also known as separation of privilege, separation of duties is a security principle which requires that the successful completion of a single task is dependent upon two or more conditions that are insufficient, individually by themselves, for completing the task.
- Example:
- Split knowledge is when a key (or password) is split into two or more pieces such that each piece is unusable by itself, and it requires two (or more) pieces be brought together to decrypt the data (or access the system).
- For example, a vendor might generate a large key and split it into (say m) multiple pieces.
- In this model a minimum of n pieces are required to recreate the key encryption key.
- The numbers n and m are configurable so that one can split the key into a dozen pieces, but only require any three of the key holders to come together to unlock the key.

Saltzer and Schroeder's Principles of Security

- **Separation of Duties**
- Example: Multi-factor authentication(MFA):
- For access to a system that supports multi-factor authentication (MFA), dual control could involve entrusting the password to one individual, and the MFA device to a different individual.

Saltzer and Schroeder's Principles of Security

- Economy of Mechanism
- Also known as 'keep it simple', if there are multiple implementations then the simplest and most easily understood implementation should be chosen.

Saltzer and Schroeder's Principles of Security

- **Complete Mediation**

- A security principle that ensures that authority is not circumvented in subsequent requests of an object by a subject, by checking for authorization (rights and privileges) upon every request for the object.
- **In other words, the access requests by a subject for an object are completely mediated/intervened every time, so that all accesses to objects must be checked to ensure that they are allowed.**
- Modern security architecture such as Zero-Trust applies this principle to both internal and external network. There is no exception to trusted network segment.

Saltzer and Schroeder's Principles of Security

- **Open Design**

- The open design security principle states that the implementation details of the design should be independent of the design itself, allowing the design to remain open while the implementation can be kept secret
- Do not rely solely upon security through obscurity(using a cryptographic algorithm that is not widely known).
- Security should depend upon keeping keys or passwords secret, not the design of the system.
- This is not to say that your security architecture should be published publicly—security through obscurity is still security, just that it ought not to be relied upon in and of itself.
- In the field of cryptography this principle was first associated with the Dutch cryptographer Auguste Kerckhoffs.
- Kerckhoffs's principle basically states that a cryptographic method ought to be secure even if everything about the system, except the key, is known by the attacker.

Saltzer and Schroeder's Principles of Security

- Least Common Mechanism

- The security principle of least common mechanisms disallows the sharing of mechanisms that are common to more than one user or process if the users or processes are at different levels of privilege. This is important when defending against privilege escalation.
- **Minimize the sharing of components between users, especially when the component must be used by all users. Such sharing can give rise to a single point of failure.**
- Example: Process isolation is one example.
- Process runs in separate memory space, direct access between processes is limited,
- The memory contents of a process cannot be read by the other processes, unless it is explicitly shared through the system calls.

Saltzer and Schroeder's Principles of Security

- **Psychological acceptability**
- A security principle that aims at maximizing the usage and adoption of the security functionality in the software by ensuring that the security functionality is easy to use and at the same time transparent to the user.
- Ease of use and transparency are essential requirements for this security principle to be effective.

Threat description: Use threat tree and subtree(explanation table)

- Threat Tree-Tampering process

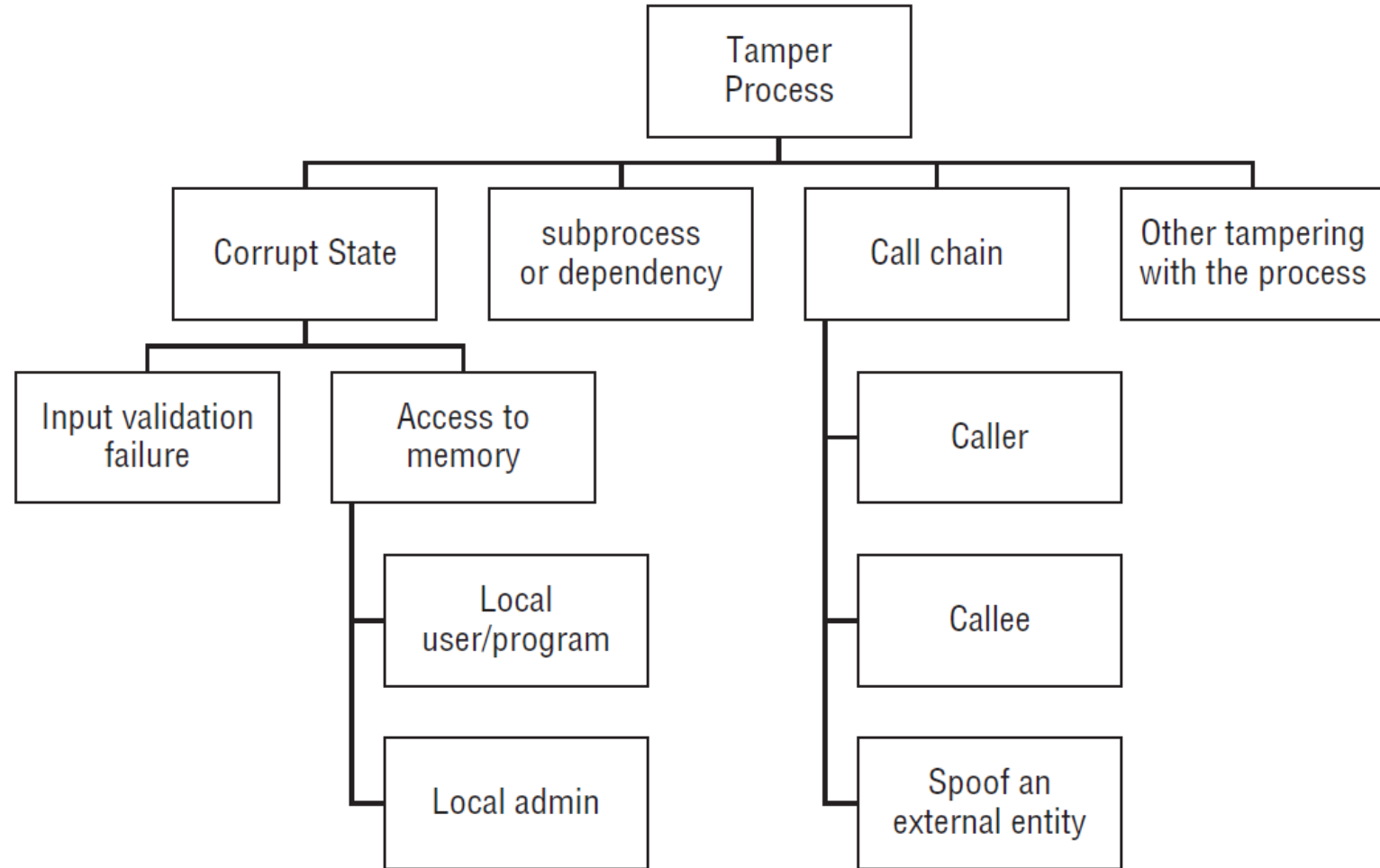


Figure B-4: Tampering with a process

Threat description

Use threat tree and subtree(explanation table)

- Explanation Table-
 - Tampering process,
 - However this description should be similar to given example in start activity.
- Therefore student may take help from following resources CVE

- 1. Use CVE for threat description
- CVE: Common Vulnerabilities and Exposures
- SANS software error: Top 25 Software Errors | SANS Institute
- <https://www.sans.org/top25-software-errors/>

Table B-4a: Corrupt State Subtree

TREE NODE	EXPLANATION	DEVELOPER MITIGATION	OPERATIONAL MITIGATION
Input validation failure	If inputs are not appropriately validated, memory corruption can result in EoP or DoS.	Carefully validate all input for the appropriate purpose.	None for the threat, sandboxing can contain impact
Access to memory (local user/program)	A user with authorized write access to memory can tamper with the process. It's important to realize that on many operating systems, this is the case for all programs the user runs. (Can you freely attach a debugger? If so, a program you're running can do the same.)	Create new account; Review shared memory permissions.	few
Access to memory (local admin)	A local admin with a debugger can be a threat (e.g., you're trying to use DRM).	Memory protection and anti-debugging schemes, generally used by DRM and other malware	separate machines

Attack Vector

- An attack vector is a segment of the entire pathway that an attack uses to access a vulnerability.
- Each attack vector can be thought of as comprising **a source of malicious content**, **a potentially vulnerable processor of that malicious content**, and **the malicious content itself**.
- An attack may involve multiple vectors in a sequence that forms the attack path to the system.
- The boundary between one vector and the next vector to another vector is that: each vector can be stopped by using a security control.
- Examples of attack vectors are:
 - Malicious web page content (content) downloaded from a web site (source) by a vulnerable web browser (processor);
 - A malicious email attachment (content) in an email client (source) rendered by a vulnerable helper application (processor);
 - A malicious email attachment (content) downloaded from an email server (source) to a vulnerable email client (processor);
 - Social engineering-based conversation (content) performed by phone from a human attacker (source) to get a username and password from a vulnerable user (processor);

Attack and attack vector

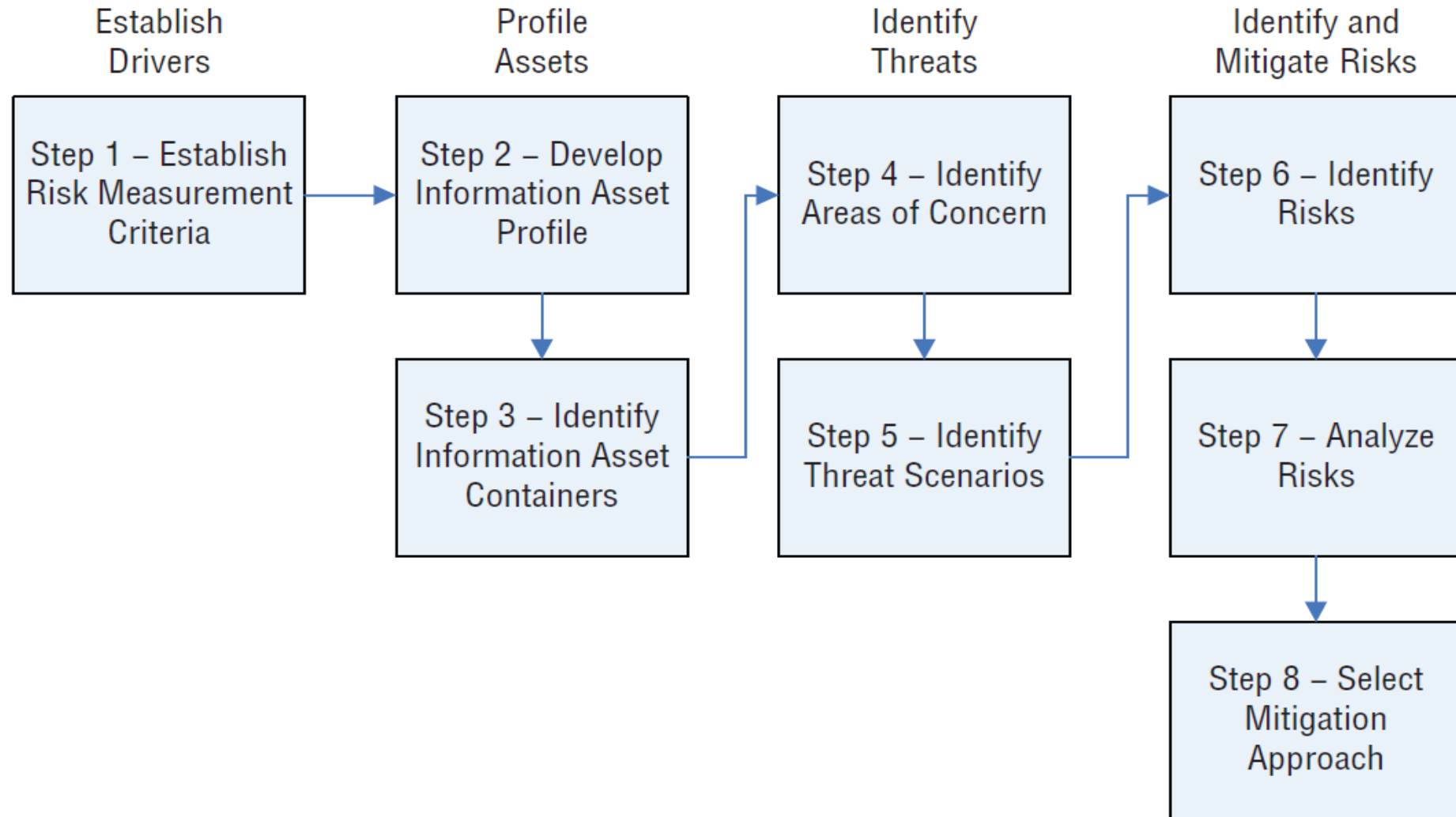
- An *attack* comprises a scenario that may occur and a single path (one or more attack vectors in sequence) that could be taken for that scenario.
- The characteristics of attacks vary widely.
- Some attack may involve exploitation of a single vulnerability using a single attack vector,
- While other attacks may involve multiple vulnerabilities and multiple attack vectors,
- Or even a single vulnerability and multiple attack vectors.
- And the vulnerabilities and attack vectors may be spread across multiple hosts, compromising one host in order to compromise another, further complicating the composition of an attack.
- Each attack vector affords an opportunity to detect and stop the attack before it goes any farther.
- Attack model: Set of all possible attack paths to a system under consideration where each path independently can break one or more security properties.
- Threat model: Attack model plus mitigation strategies of attack paths including attack vectors.

Artefact to present attack model

- Sequence diagram
- Attack description/scenario
- Attacker sources tree, Attack Tree (combined with above)
- Example Reference: Shamsul Huda, MR Islam, J Abawajy, A Cyber Risk Assessment Approach to Federated Identity Management Framework-Based Digital Healthcare System, Sensors 24 (16), 5282

Other Threat Modelling

- Operationally Critical Threat, Asset, and Vulnerability Evaluation (OCTAVE)



OCTAVE

- There are four phases of OCTAVE Allegro:
 1. Establish drivers, where the organization develops risk measurement criteria that are consistent with organizational drivers. These are qualitative risk evaluation criteria that describe the organization's acceptable levels of risk.
 2. Profile assets, where the assets that are the focus of the risk assessment are identified and profiled and the assets' containers are identified (Assets identification, valuation).
 3. Identify threats, where threats to the assets—in the context of their containers—are identified and documented through a structured process.
 4. Identify and mitigate risks, where risks are identified and analyzed based on threat information and mitigation strategies are developed to address those risks

References:

Reference:

1. <https://devguide.owasp.org/en/02-foundations/03-security-principles/>
2. Shamsul Huda, MR Islam, et al., A Cyber Risk Assessment Approach to Federated Identity Management Framework-Based Digital Healthcare System, Sensors 24 (16), 5282
3. **Shostack, Adam. Threat Modeling : Designing for Security, John Wiley & Sons, Incorporated, 2014.**
4. Guide to Data-Centric System Threat Modeling, **NIST Special Publication 800-154**
5. **Albert, C., et al. (1999 September). Operationally Critical Threat, Asset, and Vulnerability Evaluation (OCTAVE) Framework, Version 1.0. Carnegie Mellon University, Operationally Critical Threat, Asset, and Vulnerability Evaluation (OCTAVE) Framework, Version 1.0 | CMU Software Engineering Institute**